Boing Network — Operational Runbook

> ***Purpose:*** *Operations guide for running and maintaining Boing Network nodes.*
> ***References:*** *BOING-NETWORK-ESSENTIALS.md (six pillars and network essentials), BUILD-ROADMAP.md, README.md, RPC-API-SPEC.md, READINESS.md (beta checklist and quick starts)*

Network essentials (six pillars)

The network prioritizes, in order: **1. Security** → **2. Scalability** → **3. Decentralization** → **4. Authenticity** → **5. Transparency** → **6. True quality assurance**. For the full description of each pillar and design philosophy, see BOING-NETWORK-ESSENTIALS.md.

Table of Contents

1. Node Setup
2. Running a Node
3. RPC Endpoints
4. CLI Usage
5. Monitoring & Health
6. Incident Response
7. Troubleshooting
8. Testnet Operations

---

1. Node Setup

Prerequisites

- **Rust:** 1.70+ (`rustup` recommended)
- **OS:** Linux, macOS, or Windows (WSL recommended on Windows)

Build

```
cargo build --release
```

Directory Layout

Path	Description
`target/release/boing-node`	Node binary
`target/release/boing`	CLI binary
`~/.boing/` or `./data/`	Data directory (when using `--data-dir`)

---

2. Running a Node

Full Node (non-validator)

```
cargo run -p boing-node
```

Defaults: RPC on `http://127.0.0.1:8545`.

Validator Node

```
cargo run -p boing-node -- --validator --rpc-port 8545
```

Produces blocks when there are pending transactions.

With Data Directory

```
cargo run -p boing-node -- --data-dir ./boing-data --rpc-port 8545
```

---

3. RPC Endpoints

Method	Params	Description
`boing_submitTransaction`	`[hex_signed_tx]`	Submit a signed transaction
`boing_chainHeight`	`[]`	Current chain height
`boing_getBalance`	`[hex_account_id]`	Spendable balance (decimal string)
`boing_getAccount`	`[hex_account_id]`	Balance, nonce, stake (for wallets and tx building)
`boing_getBlockByHeight`	`[height]`	Block at height (u64)
`boing_getBlockByHash`	`[hex_block_hash]`	Block by hash (32 bytes hex)
`boing_getAccountProof`	`[hex_account_id]`	Merkle proof for account
`boing_verifyAccountProof`	`[hex_proof, hex_state_root]`	Verify Merkle proof
`boing_simulateTransaction`	`[hex_signed_tx]`	Simulate tx (gas, success)
`boing_registerDappMetrics`	`[hex_contract, hex_owner]`	Register dApp for incentives
`boing_submitIntent`	`[hex_signed_intent]`	Submit signed intent for solver fulfillment
`boing_faucetRequest`	`[hex_account_id]`	Testnet only: request testnet BOING (node must be started with `--faucet-enable`)

Example (curl):

```
curl -X POST http://127.0.0.1:8545/ -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"boing_chainHeight","params":[]}'
```

---

4. CLI Usage

Command	Description
`boing init [name]`	Scaffold a new dApp project
`boing dev [--port 8545]`	Start local dev chain
`boing deploy [path]`	Deploy contract or config
`boing metrics register --contract <hex> --owner <hex>`	Register contract for dApp incentives
`boing completions <shell>`	Generate shell completion (bash, zsh, fish, powershell, elvish)

Shell Completion

```
# Bash
boing completions bash > /etc/bash_completion.d/boing  # or ~/.local/share/bash-completion/completions/boing

# Zsh
boing completions zsh > ~/.zsh/completions/_boing

# Fish
boing completions fish > ~/.config/fish/completions/boing.fish
```

---

5. Monitoring & Health

Chain Height

```
curl -s -X POST http://127.0.0.1:8545/ -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"boing_chainHeight","params":[]}' | jq
```

Block Query

```
curl -s -X POST http://127.0.0.1:8545/ -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"boing_getBlockByHeight","params":[0]}' | jq
```

Logs

Set `RUST_LOG` before running:

```
RUST_LOG=info cargo run -p boing-node
# Debug: RUST_LOG=debug
# Trace: RUST_LOG=trace
```

---

6. Incident Response

For security incidents and vulnerabilities:

Step	Action
1. **Detect**	Monitor logs, alerts, community reports.
2. **Assess**	Classify severity: Low, Medium, High, Critical.
3. **Contain**	Isolate affected systems; pause if necessary.
4. **Communicate**	Notify validators, users, ecosystem per severity.
5. **Remediate**	Apply fixes; coordinate upgrades via governance if needed.
6. **Post-mortem**	Document cause, impact, and prevention.

Contacts: See SECURITY-STANDARDS.md for audit and bug bounty details.

6b. Scalability Characteristics

- **Block time:** ~2 seconds (configurable via tokenomics)
- **Throughput:** Parallel transfer batches; access-list batching reduces conflicts
- **Gas:** Fixed per tx type (Transfer, Bond, Unbond, ContractCall, ContractDeploy)
- **Batching:** Scheduler groups non-conflicting txs; transfers with disjoint access lists run in parallel

6c. Decentralization Design

- **Permissionless validation:** No whitelist; anyone with stake can validate
- **P2P discovery:** Bootnodes for bootstrap; mDNS for LAN; DHT (roadmap) for discovery
- **No central gatekeeper:** Consensus, governance, and QA pool are decentralized
- **Single-client today:** Multiple implementations encouraged for resilience

7. Troubleshooting

Node won't start

1. Ensure port 8545 (or `--rpc-port`) is free.
2. Check `RUST_LOG=debug` for errors.
3. On Windows: ensure no firewall blocking; try WSL if TCP binding fails.

Transaction not included

- Validator mode must be enabled for block production.
- Check mempool size and nonce ordering.
- Simulate first: `boing_simulateTransaction` to validate.
- **Note:** If block production or consensus fails, transactions are re-inserted into the mempool automatically so they can be retried in the next round.

RPC returns "Method not found"

- Ensure you're using the exact method name (case-sensitive).
- Params must be a JSON array (e.g. `"params": []` not `"params": {}`).

Build fails

```
cargo clean
cargo build
```

---

8. Testnet Operations

When running the **public incentivized testnet**, the following operations keep bootnodes and the faucet available. See INCENTIVIZED-TESTNET.md for the full launch checklist.

8.1 Running a bootnode

A **bootnode** is a node with a stable, publicly reachable address that other nodes use to join the network.

1. **Build:** `cargo build --release`
2. **Run with P2P and a fixed port:**

   ```
   ./target/release/boing-node \
     --p2p_listen /ip4/0.0.0.0/tcp/4001 \
     --validator \
     --rpc-port 8545 \
     --data-dir ./bootnode-data
   ```

3. **Publish the multiaddr:** Your bootnode address is `/ip4/<YOUR_PUBLIC_IP>/tcp/4001`. Ensure TCP port 4001 (and 8545 if RPC is public) is open in the firewall. Add this multiaddr to TESTNET.md §6 and to `website/src/config/testnet.ts` (or set `PUBLIC_BOOTNODES` at build time).
4. **Recommendation:** Run at least **two** bootnodes on different hosts for redundancy.

8.2 Running the faucet node

The **faucet** is an RPC method (`boing_faucetRequest`) on a node started with `--faucet-enable`. Use a dedicated node (or a node behind your public RPC) so the website faucet page can target it.

1. **Build:** `cargo build --release`
2. **Run with faucet enabled:**

   ```
   ./target/release/boing-node \
     --validator \
     --faucet-enable \
     --rpc-port 8545 \
     --data-dir ./faucet-data
   ```

   For the **public testnet**, also use `--p2p_listen` and `--bootnodes` so this node syncs with the network.
3. **Publish the RPC URL:** Point users and the website to this node's RPC (e.g. `https://testnet-rpc.boing.network/`). Set `PUBLIC_TESTNET_RPC_URL` when building the website so the faucet page defaults to this URL.
4. **Rate limit:** The faucet allows 1 request per 60 seconds per account; no extra config needed.
5. **Genesis:** The faucet account is funded at genesis with 10,000,000 testnet BOING (see `boing-node/src/faucet.rs`). Ensure all testnet nodes use the same genesis so the faucet balance exists.

8.3 Cloudflare Tunnel (testnet-rpc.boing.network)

For full setup steps, see INFRASTRUCTURE-SETUP.md. If you use Cloudflare Tunnel to expose the RPC at `https://testnet-rpc.boing.network/` :

- **"Failed to initialize DNS local resolver"** — This cloudflared log message is usually harmless. The tunnel has already registered; traffic forwarding works. It occurs when cloudflared cannot reach `region1.v2.argotunnel.com` for optional region/metrics. You can ignore it. If it bothers you, try a different DNS (e.g. 1.1.1.1) or firewall rules that allow outbound DNS.

8.4 Monitoring the testnet

- **Chain height:** Call `boing_chainHeight` on the public RPC periodically; alert if growth stalls.
- **Faucet balance:** Check the faucet account balance via `boing_getBalance` with the faucet account ID; refill or alert when low (genesis funding is 10M; 1,000 per request).
- **Bootnode reachability:** Ensure ports 4001 (P2P) and 8545 (RPC, if exposed) are reachable from the internet; use a simple TCP check or your monitoring stack.

Boing Network — Authentic. Decentralized. Optimal. Sustainable.